SYSTEM ARCHITECT ARM CORTEX A

Bài 3.2: Touchscreen Driver

Biên soạn: **Phạm Văn Vũ**

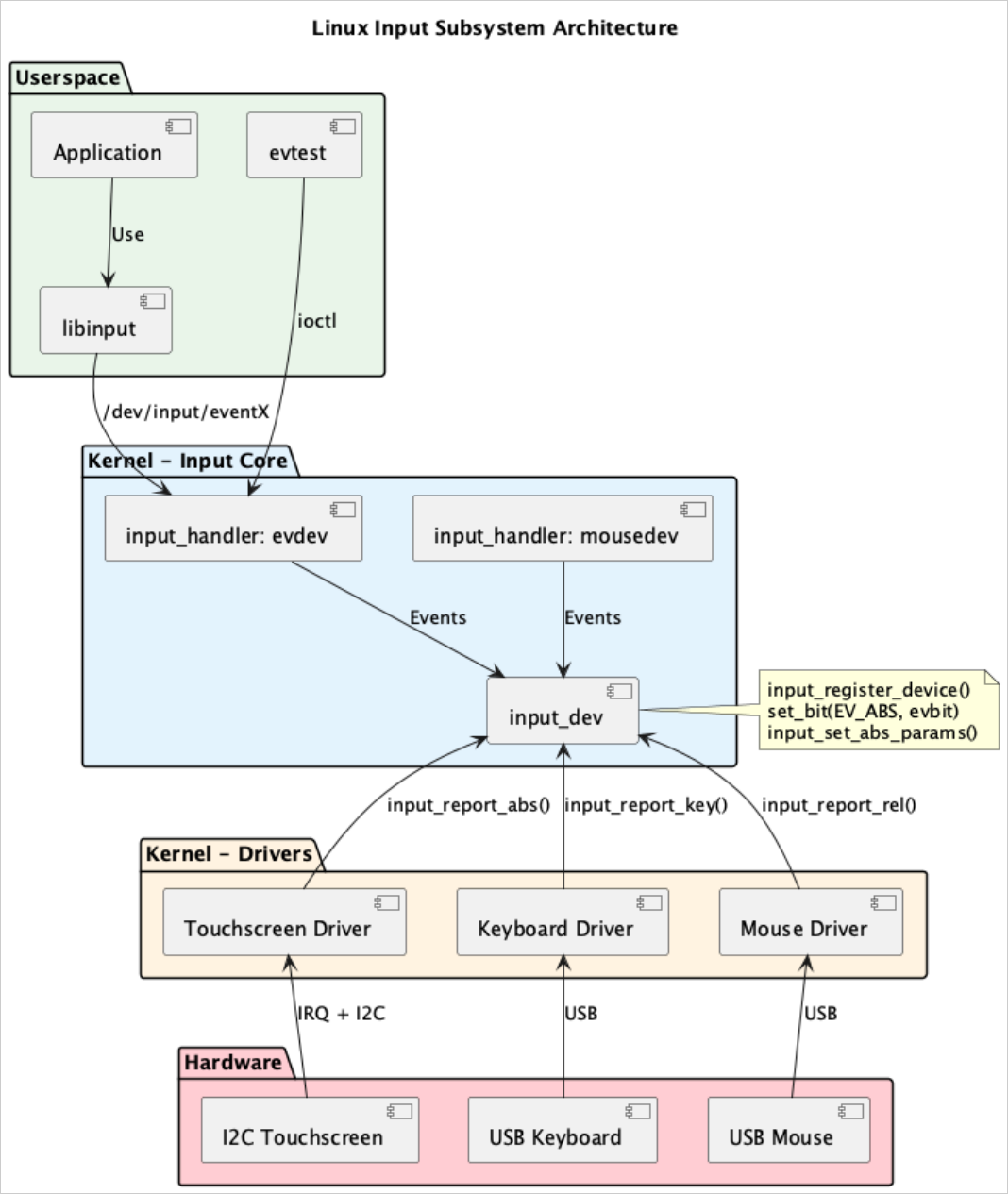
HALA Academy

Mục tiêu Bài học

Sau buổi học này, học viên sẽ có khả năng:

- Hiểu **Linux Input Subsystem**
- Viết skeleton driver cho **I2C touchscreen**
- Debug với **evtest** và **libinput**

Phần 1: Linux Input Subsystem



Hình 1: Kiến trúc Input Subsystem

1.1 Kiến trúc

Luồng dữ liệu: Driver → input_dev → evdev handler → /dev/input/eventX → Userspace

1.2 Event Types

Type	Macro	Mô tả
EV_SYN	0x00	Sync event
EV_KEY	0x01	Key/button
EV_REL	0x02	Relative (mouse)
EV_ABS	0x03	Absolute (touch)

1.3 Touch Events (EV_ABS)

Code	Macro	Mô tả
0x00	ABS_X	X position
0x01	ABS_Y	Y position
0x18	ABS_PRESSURE	Pressure
0x35	ABS_MT_POSITION_X	Multitouch X
0x36	ABS_MT_POSITION_Y	Multitouch Y

Phần 2: Driver Structure

2.1 IRQ Handler

```
static irqreturn_t my_touch_irq(int irq, void *dev_id)
{
    struct my_touch_data *data = dev_id;
    u8 buf[6];
    int x, y, touch;

    // Read touch data from I2C
    i2c_smbus_read_i2c_block_data(data->client, 0x00, 6, buf);

    touch = buf[0] & 0x01;
```

```
    x = (buf[1] << 8) | buf[2];
    y = (buf[3] << 8) | buf[4];

    if (touch) {
        input_report_abs(data->input, ABS_X, x);
        input_report_abs(data->input, ABS_Y, y);
        input_report_key(data->input, BTN_TOUCH, 1);
    } else {
        input_report_key(data->input, BTN_TOUCH, 0);
    }

    input_sync(data->input);
    return IRQ_HANDLED;
}
```

2.2 Probe Function

```
static int my_touch_probe(struct i2c_client *client)
{
    struct my_touch_data *data;
    struct input_dev *input;

    data = devm_kzalloc(&client->dev, sizeof(*data), GFP_KERNEL);
    input = devm_input_allocate_device(&client->dev);

    data->client = client;
    data->input = input;

    input->name = "My Touchscreen";
    input->id.bustype = BUS_I2C;

    // Set capabilities
    set_bit(EV_ABS, input->evbit);
    set_bit(EV_KEY, input->evbit);
    set_bit(BTN_TOUCH, input->keybit);

    input_set_abs_params(input, ABS_X, 0, 1024, 0, 0);
    input_set_abs_params(input, ABS_Y, 0, 600, 0, 0);

    // Request IRQ & Register
    devm_request_threaded_irq(&client->dev, client->irq,
                              NULL, my_touch_irq, IRQF_ONESHOT,
                              "my_touch", data);
    input_register_device(input);

    return 0;
}
```

Phần 3: Device Tree Binding

```
&i2c1 {
    status = "okay";

    touchscreen@38 {
        compatible = "vendor,my-touch";
        reg = <0x38>;
        interrupt-parent = <&pio>;
        interrupts = <7 4 IRQ_TYPE_EDGE_FALLING>;  // PH4

        touchscreen-size-x = <1024>;
        touchscreen-size-y = <600>;
    };
};
```

Phần 4: Debug và Test

4.1 evtest

```
# Install
apt install evtest

# List và monitor devices
evtest
# /dev/input/event0: My Touchscreen

evtest /dev/input/event0
# Event: type 3 (EV_ABS), code 0 (ABS_X), value 512
# Event: type 3 (EV_ABS), code 1 (ABS_Y), value 300
# Event: type 1 (EV_KEY), code 330 (BTN_TOUCH), value 1
```

4.2 Check driver status

```
# Check module
lsmod | grep my_touch

# Check input device
```

```
cat /proc/bus/input/devices

# Check DT binding
cat /sys/bus/i2c/devices/1-0038/of_node/compatible
```

Phần 5: Multitouch Support

```
// In probe
input_mt_init_slots(input, MAX_FINGERS, INPUT_MT_DIRECT);

// In IRQ handler
for (i = 0; i < num_fingers; i++) {
    input_mt_slot(input, i);
    input_mt_report_slot_state(input, MT_TOOL_FINGER, touch);
    if (touch) {
        input_report_abs(input, ABS_MT_POSITION_X, x[i]);
        input_report_abs(input, ABS_MT_POSITION_Y, y[i]);
    }
}
input_mt_sync_frame(input);
input_sync(input);
```

Phần 6: Câu hỏi Ôn tập

1. Giải thích luồng data từ driver đến userspace.
2. EV_ABS và EV_KEY khác nhau như thế nào?
3. input_sync() có chức năng gì?
4. Làm sao debug với evtest?
5. Multitouch Type B protocol là gì?

Tài liệu Tham khảo

- Kernel Input Documentation
- Multi-touch Protocol
- evtest

Yêu cầu Bài tập

- Hiểu flow: Driver → input_dev → evdev → userspace
- Build được skeleton driver module
- Test với evtest thấy events